

SE LAB-3

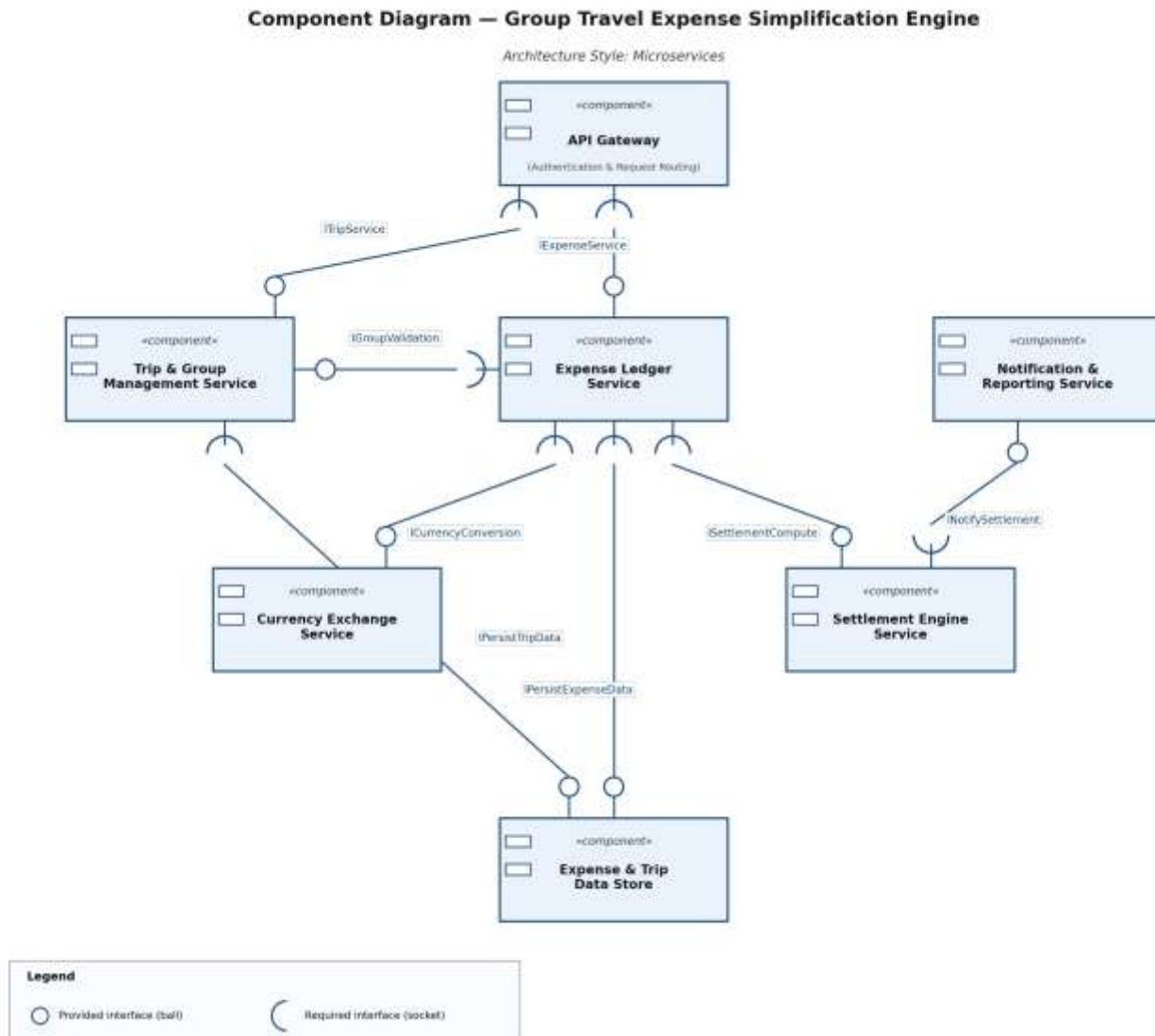
Lab 3 — Component Modelling & Architectural Pattern Selection

Vasu Tripathi SRN: PES1UG24AM918

Scenario: 37 Group Travel Expense Simplification Engine

1.Component Diagram

Lab 3 — Component Modelling & Architectural Pattern Selection



2.Architectural Justification

Group Travel Expense Simplification Engine

Architecture Selection: "We chose Microservices Architecture for the Group Travel Expense Simplification Engine."

Architectural Choice

The system is decomposed into independently deployable microservices — API Gateway, Trip & Group Management, Expense Ledger, Currency Exchange, Settlement Engine, Notification & Reporting, and a shared Expense & Trip Data Store — each owning a distinct responsibility and communicating through well-defined provided/required interfaces rather than shared internal state. This mirrors the natural boundaries already visible in the problem statement: currency conversion, debt-graph computation, and expense recording are functionally separate concerns with different load profiles, different failure modes, and different rates of change, which is exactly the situation microservices are designed to exploit.

Two Reasons for This Choice

- **Independent scalability of the settlement computation:** NFR-001 requires the min-flow debt settlement algorithm to execute in under 100 ms for groups of up to 50 travelers and 500 expenses. This is a CPU-bound graph optimization problem with a very different resource profile from simple CRUD operations like editing a trip name. Isolating it in its own Settlement Engine Service lets it be scaled horizontally, given dedicated compute, and cached independently of lighter-weight services such as trip management — without over-provisioning the entire application just to keep one algorithm fast.
- **Fault isolation around external currency data:** Daily exchange-rate conversion depends on a third-party feed outside the team's control, which can be slow, rate-limited, or briefly unavailable. Keeping the Currency Exchange Service as its own deployable unit means a stalled or failing exchange-rate call degrades only currency conversion — travelers can still log domestic expenses, view existing balances, and add group members — instead of a single shared process bringing the whole application down with it.

Security Advantage

A single API Gateway centralizes authentication and role-based authorization for the Group Traveler and Trip Admin actors (e.g., JWT/OAuth2 tokens validated once at the edge). Downstream services never talk to clients directly and only accept pre-validated, scoped requests from the gateway, so each microservice can enforce the principle of least privilege — for example, only Trip Admins are authorized to close a settlement. If any single service is compromised (such as Currency Exchange, which integrates with an external provider), the blast radius is contained to that service's own data and permissions rather than exposing the Expense Ledger, the Settlement Engine, or the underlying data store.

Performance Benefit

Because settlement computation is decoupled from the rest of the system, the Settlement Engine Service can hold in-memory balance graphs, use an algorithm implementation tuned specifically for min-cost-flow settlement, and be given dedicated resources or horizontal replicas as trip size grows. This lets it meet the sub-100 ms latency target defined in NFR-001 without competing for CPU or memory with unrelated traffic such as notification delivery or currency lookups, both of which can run on far more modest infrastructure.

Trade-offs Acknowledged

Microservices introduce operational overhead that a single-process design would not have: inter-service network calls add latency compared to an in-process function call, and the team must maintain service discovery, monitoring, and independent deployments for seven components instead of one. For this scenario, the trade-off is acceptable because the settlement algorithm's performance requirement and the currency feed's external dependency are the two biggest risks to reliability and speed, and both are precisely the risks that service isolation is best suited to contain.